torch.fx.experimental.symbolic_shapes.c

https://pytorch.org/docs/stable/generated/torch.fx.experimental.symbolic_sha

Description:

After having run fake tensor propagation and producing example_value result, traverse example_value looking for freshly bound unbacked symbols and record their paths for later. It is an error if we have allocated an unbacked SymInt but it cannot be found in example_value. (NB: this means if you have a multi-output function, you must call this on the tuple of tensor output, you cannot wait!) The peek parameter lets you check out what the bindings are without changing the affected list. This is primarily useful for ensuring unbacked_var_to_val is promptly populated when propagate_real_tensors is on. Optional[dict[sympy.core.symbol.Symbol, tuple[torch.utils._pytree.KeyEntry, …]]]

Function Signature

```
torch.fx.experimental.symbolic_shapes.compute_unbacked_bindi
ngs(shape_env, example_value, old_example_value=None,
peek=False)[source]#

Return type
```

Returns:

Optional[dict[sympy.core.symbol.Symbol, tuple[torch.utils._pytree.KeyEntry, ...]]]

Detailed Documentation

Documentation

After having run fake tensor propagation and producing example_value result, traverse example_value looking for freshly bound unbacked symbols and record their paths for later. It is an error if we have allocated an unbacked SymInt but it cannot be found in example_value. (NB: this means if you have a multi-output function, you must call this on the tuple of tensor output, you cannot wait!)

The peek parameter lets you check out what the bindings are without changing the affected list. This is primarily useful for ensuring unbacked_var_to_val is promptly populated when propagate_real_tensors is on.

Optional[dict[sympy.core.symbol.Symbol, tuple[torch.utils._pytree.KeyEntry, …]]]